

- a. Gemini and ChatGPT
- b. Gemini on Website and ChatGPT on windows app.
- c.
 - The program will prioritize emails for a busy company CEO.
 - You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
 - You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
 - You must create functions from scratch. Do not include pre-existing heap modules.

Test File format:

Command	Description
EMAIL <sender category>, <subject line>, <date>	The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows: • Boss – read first • Subordinate – read next • Peer – read next • ImportantPerson – read next • OtherPerson – read last If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint. EMAIL is followed by space. The rest of the fields are delimited. Assume <sender category> is one of the five strings listed above. Assume <subject line> is a string which may contain spaces, but not commas Assume <date> is in the format: MM-DD-YYYY
NEXT	Next email for the CEO to read. Display the information on the terminal in the following format: Sender: Subject: Date:
READ	CEO has read email and dealt with it
COUNT	Display current count of unread emails

- The user will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

Sample Test File:

```
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
```

Your program should display this:

```
There are 6 emails to read.

Next email:
    Sender: Boss
    Subject: Never Mind
    Date: 01-03-2025

Next email:
    Sender: Boss
    Subject: Important
    Date: 12-20-2024

There are 4 emails to read.
```

write in C

d. Gemini

```
/*
Name of the program = EECS 348 Assignment 2
Description =
Inputs=
Outputs =
Collaborators =
Other sources =
Author = Aditya Kulkarni
Creation date = 9/17/2026
Revision date = 9/17/2026
Revision =
*/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

// --- Structure Definitions ---

typedef struct {
    char sender[32];
    char subject[128];
    char date[16]; // MM-DD-YYYY
    int priority;
    long raw_date; // YYYYMMDD for easy chronological comparison
} Email;

typedef struct {
    Email* array;
    int size;
    int capacity;
} MaxHeap;

// --- Helper Functions ---

// Determine sender priority (higher number = higher priority)
int get_priority(const char* sender) {
    if (strcmp(sender, "Boss") == 0) return 5;
```

```

    if (strcmp(sender, "Subordinate") == 0) return 4;
    if (strcmp(sender, "Peer") == 0) return 3;
    if (strcmp(sender, "ImportantPerson") == 0) return 2;
    if (strcmp(sender, "OtherPerson") == 0) return 1;
    return 0; // Unknown category fallback
}

// Convert MM-DD-YYYY to an integer YYYYMMDD
long parse_date(const char* date_str) {
    int m, d, y;
    if (sscanf(date_str, "%d-%d-%d", &m, &d, &y) == 3) {
        return y * 10000 + m * 100 + d;
    }
    return 0;
}

// Compare two emails to determine which has higher priority
bool is_higher_priority(Email a, Email b) {
    // 1. Check sender category priority
    if (a.priority > b.priority) {
        return true;
    }
    // 2. Tie-breaker: Check date (newest email read first)
    if (a.priority == b.priority) {
        return a.raw_date > b.raw_date;
    }
    return false;
}

// --- MaxHeap Operations ---

MaxHeap* create_heap(int initial_capacity) {
    MaxHeap* heap = (MaxHeap*)malloc(sizeof(MaxHeap));
    heap->capacity = initial_capacity;
    heap->size = 0;
    heap->array = (Email*)malloc(sizeof(Email) * initial_capacity);
    return heap;
}

void swap(Email* a, Email* b) {

```

```

    Email temp = *a;
    *a = *b;
    *b = temp;
}

void heapify_up(MaxHeap* heap, int index) {
    int parent = (index - 1) / 2;
    while (index > 0 && is_higher_priority(heap->array[index],
heap->array[parent])) {
        swap(&heap->array[index], &heap->array[parent]);
        index = parent;
        parent = (index - 1) / 2;
    }
}

void heapify_down(MaxHeap* heap, int index) {
    int max_index = index;
    int left = 2 * index + 1;
    int right = 2 * index + 2;

    if (left < heap->size && is_higher_priority(heap->array[left],
heap->array[max_index])) {
        max_index = left;
    }

    if (right < heap->size && is_higher_priority(heap->array[right],
heap->array[max_index])) {
        max_index = right;
    }

    if (max_index != index) {
        swap(&heap->array[index], &heap->array[max_index]);
        heapify_down(heap, max_index);
    }
}

void insert_email(MaxHeap* heap, Email email) {
    // Resize array if capacity is reached
    if (heap->size == heap->capacity) {
        heap->capacity *= 2;
        heap->array = (Email*)realloc(heap->array, sizeof(Email) *
heap->capacity);
    }
}

```

```

    }
    heap->array[heap->size] = email;
    heapify_up(heap, heap->size);
    heap->size++;
}

void extract_max(MaxHeap* heap) {
    if (heap->size <= 0) {
        return; // Empty queue, nothing to extract
    }
    heap->array[0] = heap->array[heap->size - 1];
    heap->size--;
    heapify_down(heap, 0);
}

Email* peek_max(MaxHeap* heap) {
    if (heap->size <= 0) {
        return NULL;
    }
    return &heap->array[0];
}

// --- Main Execution ---

int main() {
    MaxHeap* inbox = create_heap(10);
    char line[256];

    // Read commands line by line from stdin (redirect test file in
    // via <)
    while (fgets(line, sizeof(line), stdin)) {
        // Handle EMAIL command
        if (strncmp(line, "EMAIL ", 6) == 0) {
            char* payload = line + 6;
            char* sender = strtok(payload, ",");
            char* subject = strtok(NULL, ",");
            char* date = strtok(NULL, ",\r\n"); // Extract date and
            strip_newlines

            if (sender && subject && date) {

```

```

        Email new_email;
        strcpy(new_email.sender, sender);
        strcpy(new_email.subject, subject);
        strcpy(new_email.date, date);
        new_email.priority = get_priority(sender);
        new_email.raw_date = parse_date(date);

        insert_email(inbox, new_email);
    }
}

// Handle NEXT command
else if (strncmp(line, "NEXT", 4) == 0) {
    Email* next_email = peek_max(inbox);
    if (next_email != NULL) {
        printf("Next email:\n");
        printf("Sender: %s\n", next_email->sender);
        printf("Subject: %s\n", next_email->subject);
        printf("Date: %s\n", next_email->date);
    }
}

// Handle READ command
else if (strncmp(line, "READ", 4) == 0) {
    extract_max(inbox);
}

// Handle COUNT command
else if (strncmp(line, "COUNT", 5) == 0) {
    printf("There are %d emails to read.\n", inbox->size);
}

}

// Free allocated memory
free(inbox->array);
free(inbox);

return 0;
}

```

ChatGPT

```
#include <stdio.h>
```

```

#include <stdlib.h>
#include <string.h>

/*
    Priority order:
    Boss          = 5
    Subordinate    = 4
    Peer           = 3
    ImportantPerson = 2
    OtherPerson    = 1
*/

typedef struct {
    char sender[50];
    char subject[256];

    int month;
    int day;
    int year;

    int priority;
} Email;

typedef struct {
    Email *emails;
    int size;
    int capacity;
} MaxHeap;

/* -----
Utility Functions
----- */

int getPriority(const char *sender) {
    if (strcmp(sender, "Boss") == 0)
        return 5;
    else if (strcmp(sender, "Subordinate") == 0)
        return 4;
    else if (strcmp(sender, "Peer") == 0)

```

```

        return 3;
    else if (strcmp(sender, "ImportantPerson") == 0)
        return 2;
    else
        return 1;
}

/*
Returns 1 if email a should be ABOVE email b in the heap.
*/
int higherPriority(Email a, Email b) {
    if (a.priority > b.priority)
        return 1;

    if (a.priority < b.priority)
        return 0;

    /*
    Same sender category:
    newest email comes first.
    */
    if (a.year > b.year)
        return 1;

    if (a.year < b.year)
        return 0;

    if (a.month > b.month)
        return 1;

    if (a.month < b.month)
        return 0;

    if (a.day > b.day)
        return 1;

    return 0;
}

```

```

void swapEmails(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

/* -----
   MaxHeap Functions
   ----- */

void initializeHeap(MaxHeap *heap) {
    heap->capacity = 10;
    heap->size = 0;

    heap->emails = malloc(heap->capacity * sizeof(Email));

    if (heap->emails == NULL) {
        printf("Memory allocation failed.\n");
        exit(1);
    }
}

void resizeHeap(MaxHeap *heap) {
    heap->capacity *= 2;

    Email *newArray =
        realloc(heap->emails, heap->capacity * sizeof(Email));

    if (newArray == NULL) {
        printf("Memory allocation failed.\n");
        free(heap->emails);
        exit(1);
    }

    heap->emails = newArray;
}

```

```
/*  
    Move an inserted email upward until the MaxHeap property  
    is restored.  
*/
```

```
void heapifyUp(MaxHeap *heap, int index) {  
    int parent;  
  
    while (index > 0) {  
        parent = (index - 1) / 2;  
  
        if (higherPriority(heap->emails[index],  
                           heap->emails[parent])) {  
  
            swapEmails(&heap->emails[index],  
                       &heap->emails[parent]);  
  
            index = parent;  
        } else {  
            break;  
        }  
    }  
}
```

```
/*  
    Move the root downward after removing it.  
*/
```

```
void heapifyDown(MaxHeap *heap, int index) {  
    while (1) {  
        int left = 2 * index + 1;  
        int right = 2 * index + 2;  
        int largest = index;  
  
        if (left < heap->size &&  
            higherPriority(heap->emails[left],  
                           heap->emails[largest])) {  
            largest = left;  
        }  
    }
```

```

        if (right < heap->size &&
            higherPriority(heap->emails[right],
                           heap->emails[largest])) {
            largest = right;
        }

        if (largest != index) {
            swapEmails(&heap->emails[index],
                       &heap->emails[largest]);

            index = largest;
        } else {
            break;
        }
    }
}

/*
    Insert an email into the MaxHeap.
*/
void insertEmail(MaxHeap *heap, Email email) {
    if (heap->size == heap->capacity) {
        resizeHeap(heap);
    }

    heap->emails[heap->size] = email;

    heapifyUp(heap, heap->size);

    heap->size++;
}

/*
    Return the highest-priority email without removing it.
*/
Email *peekEmail(MaxHeap *heap) {
    if (heap->size == 0) {
        return NULL;
    }
}

```

```

    }

    return &heap->emails[0];
}

/*
    Remove the highest-priority email from the heap.
*/
void removeEmail(MaxHeap *heap) {
    if (heap->size == 0) {
        return;
    }

    heap->emails[0] = heap->emails[heap->size - 1];

    heap->size--;

    if (heap->size > 0) {
        heapifyDown(heap, 0);
    }
}

/* -----
    Parsing
    ----- */

Email createEmail(char *line) {
    Email email;

    char *firstComma;
    char *secondComma;

    char senderPart[50];
    char datePart[50];

    /*
        Find the two commas:
        EMAIL Sender,Subject,Date
    */

```

```

*/
firstComma = strchr(line, ',');

if (firstComma == NULL) {
    printf("Invalid EMAIL command.\n");
    exit(1);
}

secondComma = strchr(firstComma + 1, ',');

if (secondComma == NULL) {
    printf("Invalid EMAIL command.\n");
    exit(1);
}

/*
    Get sender.
    Skip "EMAIL ".
*/
sscanf(line + 6, "%49[^\,]", senderPart);

strcpy(email.sender, senderPart);

/*
    Get subject.
*/
int subjectLength = (int)(secondComma - firstComma - 1);

strncpy(email.subject,
        firstComma + 1,
        subjectLength);

email.subject[subjectLength] = '\0';

/*
    Get date.
*/
strcpy(datePart, secondComma + 1);

sscanf(datePart, "%d-%d-%d",

```

```

        &email.month,
        &email.day,
        &email.year);

    email.priority = getPriority(email.sender);

    return email;
}

/* -----
Main
----- */

int main(void) {
    MaxHeap heap;
    char line[400];

    initializeHeap(&heap);

    while (fgets(line, sizeof(line), stdin) != NULL) {

        /*
         Remove newline character.
        */
        line[strcspn(line, "\n")] = '\0';

        /*
         Ignore blank lines.
        */
        if (strlen(line) == 0) {
            continue;
        }

        /* -----
        EMAIL command
        ----- */
        if (strncmp(line, "EMAIL ", 6) == 0) {

```

```

        Email email = createEmail(line);

        insertEmail(&heap, email);
    }

    /* -----
       NEXT command
       ----- */
    else if (strcmp(line, "NEXT") == 0) {

        Email *email = peekEmail(&heap);

        /*
           NEXT only displays the highest-priority
           email. It does NOT remove it.
        */
        if (email != NULL) {
            printf("Next email:\n");
            printf("Sender: %s\n", email->sender);
            printf("Subject: %s\n", email->subject);
            printf("Date: %02d-%02d-%04d\n",
                email->month,
                email->day,
                email->year);
        }
    }

    /* -----
       READ command
       ----- */
    else if (strcmp(line, "READ") == 0) {

        /*
           READ removes the highest-priority email.
           Nothing is displayed.
        */
        removeEmail(&heap);
    }

```

```

/* -----
COUNT command
----- */
else if (strcmp(line, "COUNT") == 0) {

    printf("There are %d emails to read.\n", heap.size);
}

}

free(heap.emails);

return 0;
}

```

- e. **Correctness** - Both programs get the main heap logic right . The difference is how they handle bad input. Gemini just quietly ignores a bad line and moves on, without telling you anything went wrong. ChatGPT stops the whole program and shows an error message when something's wrong with the input which is better at catching problems, but harsher since one bad line stops everything. Also, ChatGPT treats "unknown sender" and "OtherPerson" as the same priority, while Gemini keeps them separate. Gemini is more correct here if they're supposed to be different.

Execution Time- Basically the same speed for both. Adding or removing an email is fast in both versions no matter how many emails you have. ChatGPT does a tiny bit more work when comparing dates since it checks month, day, and year one at a time instead of one combined number like Gemini does, but this difference is too small to actually notice.

Space complexity- Also basically equal. Both grow their storage smartly (doubling in size instead of growing one at a time), so neither wastes memory. Gemini uses a tiny bit less memory per email since it stores the date as one number instead of three separate numbers like ChatGPT, but it's a small difference that doesn't really matter.

Maintainability- Gemini's code is shorter and a little easier to read top to bottom, since there's less "extra" handling code getting in the way. But that shortness comes at a cost that it skips checking whether memory allocation actually worked, so if that ever fails, the program would just break unexpectedly instead of giving a clear reason why. **ChatGPT's**

code would be a bit easier to hand off. It has clear checks in place. Both could use more comments.

f. Chat GPT version

- i. It handles errors better as it checks if memory allocation works and stops with a clear message if something is wrong, instead of crashing without explanation.
- ii. It cleans up messy input (removes extra blank lines and stray characters) before processing,
- iii. It's stricter about bad input like if an EMAIL line is missing a comma, it tells you right away instead of silently skipping it, which is easier to notice and fix during testing.
- iv. These extra checks make it easier for someone else (or future-you) to read, debug, and extend the code later.

- g. **Correctness-** OtherPerson" and any unknown sender name used to be treated as the same priority — borrowing from Gemini, "OtherPerson" now has its own separate check in getPriority(), so unknown senders correctly fall to a lower priority instead of being lumped together with OtherPerson.

Execution Time- Nothing needed here — inserting and removing emails is already fast, and this won't slow down even with a lot of emails.

Space Complexity- Nothing needed here either — memory grows in a smart, expected way, and the array doubling avoids wasting time on constant resizing.

Maintainability- Already solid, thanks to the error checks and input cleanup. The only small addition would be more in-line comments, so someone new to the code can follow each step without needing to trace through the logic themselves.